

REPORTE DE EVALUACIÓN TÉCNICA

Por: Rayas Calderón Johann Haziél
Alumno de FES Cuautitlán

Proyecto: Sistema de Control y Gestión Documental

(Revisor Técnico) **Framework/Lenguaje:** PHP Nativo, MySQL (PDO), JavaScript Vanilla, Bootstrap 5.3

1. Resumen Ejecutivo

El sistema analizado es una solución funcional y robusta orientada a resolver un problema crítico de la administración pública: el seguimiento y trazabilidad de oficios. El desarrollo demuestra un dominio claro de la lógica de negocio, manipulación de bases de datos relacionales y control de estados (Bandeja, Revisión, Concluido).

2. Fortalezas Técnicas y Buenas Prácticas

Durante la auditoría del código, se identificaron excelentes prácticas de desarrollo:

- **Seguridad contra Inyección SQL:** El sistema no concatena variables directamente en las consultas. En su lugar, utiliza el motor de **PDO (PHP Data Objects)** con sentencias preparadas (`prepare` y `execute`) en la totalidad de sus módulos. Esto blindará a la aplicación contra ataques de inyección.
- **Validación Estricta de Archivos (MIME Type):** Es un acierto de seguridad avanzado no confiar en la extensión del archivo subido. El uso de `finfo_open(FILEINFO_MIME_TYPE)` para verificar los bytes reales y garantizar que sea un `application/pdf` auténtico evita la subida de scripts maliciosos disfrazados.
- **Gestión de Permisos (RBAC):** La implementación de un Control de Acceso Basado en Roles es muy sólida. El sistema decide qué renderizar (y qué procesar) verificando el `$_SESSION['rol']`, aislando funciones destructivas o de captura exclusiva para Coordinación/Administración y limitando a los JUDs y Observadores.
- **Protección de Integridad de Datos:** La validación agregada para consultar la existencia previa de un "Número de Ingreso" antes de hacer un `INSERT` garantiza que la base de datos no sufra de folios duplicados, protegiendo la integridad referencial del sistema.

- **Criptografía Segura:** Para la creación de credenciales, el sistema emplea la función nativa `password_hash()` con los algoritmos actualizados de PHP (`PASSWORD_DEFAULT`), lo cual es el estándar actual de la industria para proteger contraseñas.

3. Áreas de Oportunidad y Refactorización (Deuda Técnica)

Como en todo proyecto en crecimiento, existen oportunidades para escalar y mejorar la base del código:

- **Acoplamiento (Falta de Modelo-Vista-Controlador):** Actualmente, la lógica de la base de datos (SQL), el procesamiento de variables (PHP) y la interfaz gráfica (HTML/CSS) conviven en los mismos archivos. A futuro, separar esto en un patrón MVC facilitaría el mantenimiento.
- **Manejo de Excepciones en Producción:** En el bloque `catch` al subir documentos, se utiliza `exit("Error de carga seguro: " . $e->getMessage());`. En un entorno de producción, mostrar errores nativos de la base de datos al usuario final es un riesgo. Se recomienda registrar el error en un archivo *Log* y mostrar un mensaje amigable al usuario.

4. Diagnóstico de Calidad: Calibración y Testing (Nota del Desarrollador)

A pesar de que la estructura lógica del sistema es congruente y el flujo principal (*Happy Path*) se ejecuta sin errores, comparto plenamente tu postura analítica: **considero que le faltan pruebas exhaustivas.**

Un sistema de esta magnitud, que maneja información institucional crítica, no puede depender únicamente de pruebas manuales. Para asegurar su estabilidad empresarial, es imperativo someter el código a las siguientes fases de testing exhaustivo:

1. **Pruebas Unitarias (Unit Testing):** Faltan *scripts* automatizados (usando herramientas como PHPUnit) que prueben funciones aisladas, por ejemplo, comprobar matemáticamente que el algoritmo de JavaScript para calcular los días hábiles jamás asigne un vencimiento en un fin de semana, sin importar el mes o los años bisiestos.
2. **Pruebas de Concurrencia y Estrés:** Aunque el sistema bloquea folios duplicados validando en PHP, carecemos de pruebas de estrés para evaluar *Race Conditions*. ¿Qué sucede si dos usuarios intentan registrar exactamente el mismo folio de respuesta en el mismo milisegundo? Se necesita validar cómo se comporta el bloqueo transaccional de MySQL bajo alta demanda.

-
3. **Pruebas de Seguridad en el Frontend (Pentesting):** No se han realizado pruebas exhaustivas de *Cross-Site Scripting* (XSS). Es necesario inyectar cadenas maliciosas (ej. `<script>alert(1)</script>`) en campos como *Autoridad Emisora* o *Asunto* para verificar que las funciones de renderizado, como `htmlspecialchars()`, estén cubriendo absolutamente todas las salidas de datos en las tablas.

Conclusión General: El sistema tiene bases de nivel intermedio-avanzado y cumple sobresalientemente con los requerimientos funcionales. Con la futura implementación de una batería de pruebas exhaustivas, pasará de ser un excelente proyecto funcional a un software de nivel *Enterprise* listo para despliegues masivos.